A05a · A Record That Owns Its Verbs — Practice A2 · 9618 §20.1

Name: _____ Due: _____

OOP appears on **every released Paper 4** — 32 out of 32 — and on Paper 3 as terminology and as pseudocode. So both halves here are exam halves. **Questions 1–6 are written, on this sheet. Questions 7–10 are code, in `A05a_Practice.py`**. One set of numbers across both.

1. Their words, not yours. Complete the table. [4]

OOP term	description
	A method that accesses the value of a property.
	A method that changes the value of a property.
`Object`	
`Method`	

2. Write the class — ours, and it says so. A courier's tracking program uses Object-Oriented Programming to design a class called **`Parcel`**. [5]

The class has the **STRING** attribute **`Address`** and the **INTEGER** attribute **`Weight`** (in grams), together with their getters and setters. It also has a **constructor with two parameters** that sets the values of these attributes.

Write **pseudocode** statements to declare the **`Parcel`** class, its attributes and the constructor. (You do not need to write out the getters and setters — say where they would go.)

3. Four things wrong. A candidate wrote this on **Paper 3**. The logic is fine; the dialect is not. [4]

```
public class Pet {
   private Name : STRING
   public Pet(GivenName : STRING)
      Name = GivenName
   end
}
```

Identify **four** things that would not be accepted, and give the correction for each.

1

2

3

4

4. From a description to a design. [4]

> A gym tracks its members and its classes. A **member** has a name, a membership number and a join date. A **class** has a title, a start time and a maximum size. A member can **book** a place on a class and can **cancel** a booking. *No class may be booked beyond its maximum size.*

(a) Name the **two classes**, and for each give **two attributes** and **one method**. [3]

class	two attributes	one method

(b) *"No class may be booked beyond its maximum size"* is a noun phrase, but it should **not** become a class. Where in your design does that rule belong, and why? [1]

5. Why bother making it private? A student says: *"Getters and setters are just extra typing. Make the attribute public and use it directly."* [3]

Give **three** things a public attribute cannot do that a getter or a setter can.

1

2

3

6. Three objects, how many of each? A program creates three **`Pet`** objects from one **`Pet`** class. [3]

(a) How many copies of the attribute **`Name`** exist? [1]

(b) How many copies of the method **`Birthday()`** exist? [1]

(c) A fourth pet is created. Explain what happens in memory that did **not** happen when the class itself was written. [1]

💻 **On the computer — `A05a_Practice.py`**

Questions 7 and 8 are **ours, and say so** — a class in the exact shape the OOP question takes on every released Paper 4, marked on the bullets those schemes use. The class table is in the file.

7	**`Drone`** — the class, its five attributes and its constructor	**[4]**
8	the five **`Get…()`** methods	**[3]**
9	**`SetColour`** — a setter that refuses a colour that is not on the list	**[4]**
10	🍦 **`DronesMade`** — one counter shared by every object, and names that are not	**[2]**

Written 23 + code 13 = **36 marks**.

🕳️ **Want to dig?** Question 6(b) has a tidy answer — *one* — and a much better question hiding behind it. If every object shares one copy of `Birthday()`, then when `tom.Birthday()` runs, **how does that one shared copy know it is Tom's age it should be adding to and not Jerry's?** Find out what the word `this` means in Java (or `self` in Python), and write one sentence explaining where it comes from. It is the answer to a question almost nobody thinks to ask.

🚀 **Above and beyond — not in the total.** Your `Drone` class counts every drone ever built. A rival design keeps the count in a *separate* class called `Fleet` that holds a list of drones. Give one thing each design does better, and say which one Cambridge's own mark schemes would recognise as "encapsulation" — and why the other is closer to how real fleets are written.

Additional space. If you need more room for any answer, continue here and write the question number in the margin beside it.